Islamic University of Technology

Lab 6: Iteration Statements

CSE 4108 : Structured Programming Lab

Section 1B

MIA, NFS, IAC, ASH

Octber 08, 2024

1 Lab Tasks

1.1 I am Small, Don't Hit Me

Write a C program that continuously takes number as input from the user and stops only when the user enters `0` or a negative number. Then the program displays the smallest positive number (last 0 or negative number do not count) so far. If the first input is `0` or negative, the program displays `NONE`.

[Sample Program]
Input 1

```
Enter an integer: 30
Enter an integer: 3.25
Enter an integer: 45.55
Enter an integer: 1.2
Enter an integer: 0.5
Enter an integer: 55
Enter an integer: 0
```

Output 1

```
Smallest number so far: 0.5
```

Input 2

```
Enter an integer: -5
```

Output 2

```
Smallest number so far: NONE
```

1.2 Tree for True

Trents are mythical creatures, living in forests. Mr. Root is an honorable trent, who aims at proper reforestation to save the world from green house effect. The local authority is also highly supportive of Mr. Root, who has in-depth knowledge about where to plant which tree. However, the only sentence that Mr Root can speak is, *"I am Root"*. As a result, he cannot inform the authority about how high each tree should be given a certain area for the proper growth of forest. This is why he has taken the decision to hire a professional computer engineer for developing a program for him to visualize each tree.

Write a program in C that would take an integer, the height of the tree as input and print out a tree like the sample output. Remember, the tree will always have a single stump. The minimum height of the tree is 1. In that case, it is just an acorn.

[Sample Program]
Input 1

```
Enter height of tree: 1
```

Output 1

```
*
```

Input 2

```
Enter height of tree: 2
```

Output 2

```
*
*
```

Input 3

```
Enter height of tree: 3
```

Output 3

```
 *
***
 *
```

Input 4

```
Enter height of tree: 6
```

Output 4

```
    *
   ***
  *****
 *******
*********
    *
```

[Hint] *Try to understand the pattern of printing, count based on the input value, how many spaces and how many asterisks are there in each line. Once you derive the relation, the printing should be easy. You need to use loop in loop to achieve the desired output. You might want to use a separate loop for printing the stump.*

1.3 Not GCD Again

It is a famous C task to find the GCD or Greatest Common Divisor of two integers.

Write a C program that would take two integers as input and find out their GCD.

[Sample Program]
Input

```
Enter Two Integers: 15 35
```

Output

```
GCD of 15 and 35 is: 5
```

[Hint] *The classic algorithm for computing the GCD, known as Euclid's algorithm, goes as follows:*

(A) Let m and n be variables containing the two numbers

(B) If n is 0, then stop: m contains the GCD

(C) Otherwise, compute the remainder when m is divided by n

(D) Copy n into m and copy the remainder into n

(E) Repeat the process, starting with testing whether n is 0

1.4 Cut it Down

Write a C program to read a fraction as input from the user and then display its reduced form by reducing it to the lowest form.

[Sample Program]
Input 1

```
Enter a fraction: 5/15
```

Output 1

```
Reduced fraction: 1/3
```

Input 2

```
Enter a fraction: 4/6
```

Output 2

```
Reduced fraction: 2/3
```

[Hint] *Task 1.3 is the hint.*

1.5 Primal Instinct

Write a C program that would take a positive integer (greater than or equal to 2) as input and print out all the prime numbers until that integer (inclusive) and the count of primes.

[Sample Program]
Input

```
Enter limit range (inclusive) : 29
```

Output

```
Primes until 29: 2 3 5 7 11 13 17 19 23 29
Prime count until 29: 10
```

[Hint] *Utilize the prime example in the slide of theory class. You can use the slide. Loop the prime check for all the numbers in the given range.*

1.6 Experimental Exponential

The value of the mathematical constant e can be expressed as an infinite series:

$$e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + ...$$

Write a program that approximates e by computing the value of:

$$e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + ... + \frac{1}{n!}$$

Where n is an integer taken as input from the user. Use `float` for the calculations, **not** `double`.

[Sample Program]
Input

```
Enter the value of n: 10
```

Output

```
Approximate e: 2.718282
```

[Hint] *Same kind of logic as Task 1.5.*